COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010: Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	10 / 15 / 26	**DATE SUBMITTED:**	10 / 15 / 26
NAME:	Fabonan, Kurt Lorenz C. Santos, Paterno Miguel R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 13.1: Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

Fabonan

Screenshot	
Analysis	The code makes it to where it makes one thread wait until another one finishes, as seen by the join(), which causes the main thread, the one in the program, to stop until the t1 thread, the function completes its job.

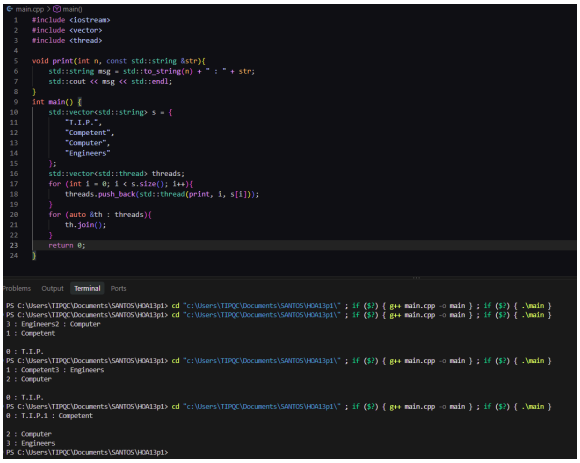
Santos	
Screenshot	
Analysis	The code prints two threads at the same time in a way that is linear to the code, following a first-in, first out.

Table 13-1. Simple One-Threaded Example

Fabonan

Screenshot	
Analysis	The code is a multithreading concurrent that basically makes the single thread constantly switch other threads. This makes it to where the output is completely random due to it choosing what thread to pick to work on randomly.

Santos

Screenshot	

Analysis	It prints multiple threads of strings at once and the order of the output can be randomized as it gets printed.

Table 13-2. Multithreaded Example

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

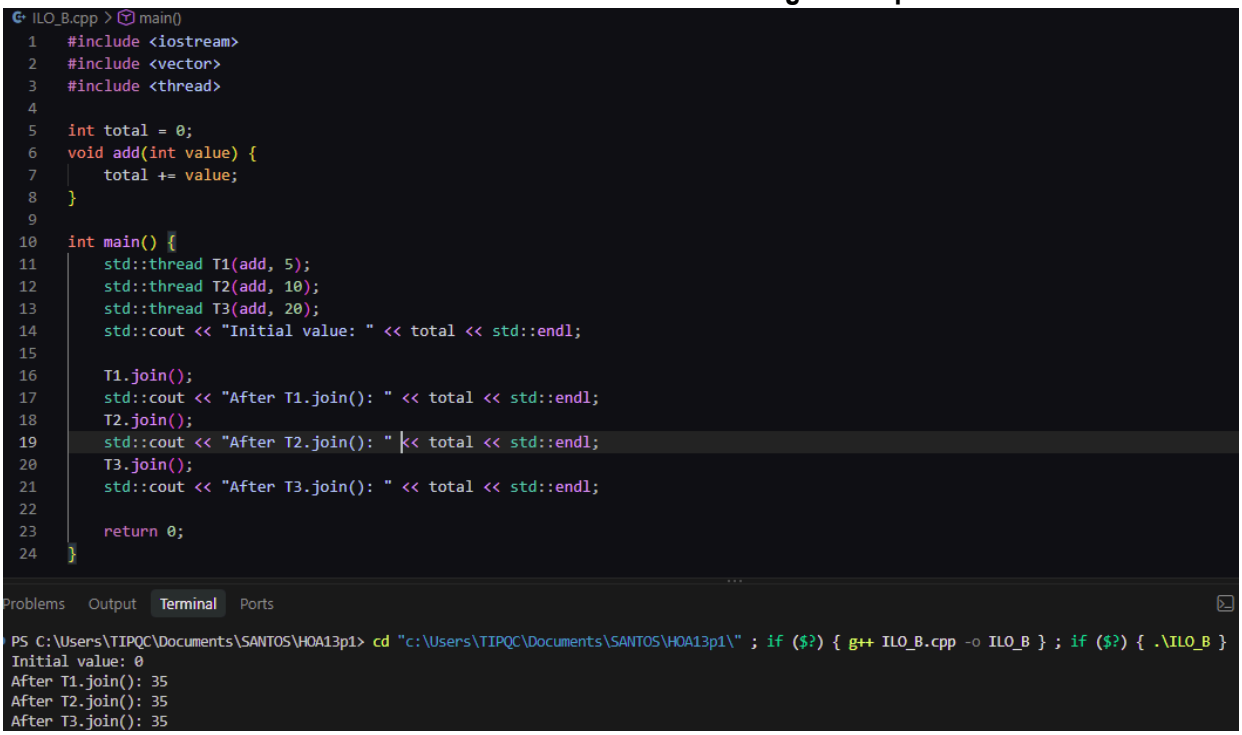
1. **Write a definition of multithreading and its advantages/disadvantages.**
   Multithreading is a programming execution and technique that splits a task into multiple parts, allowing them to run concurrently with each other instead of sequentially, while using the same memory and resources. This allows programs to be more flexible while running different things instead of waiting on another running operation, which also increases speed, and reduces need for more resources as they are shared, not duplicated. However, multithreading is much harder to program and debug, is much harder to work around with if programs use dependencies, and can oftentimes cause the program to desynchronize. It can help independent tasks run simultaneously when the programming language and system support parallel execution. But it can also be complex when managing multiple threads and consume memory along with processing time.

2. **Rationalize the use of multithreading by providing at least 3 use-cases.**
   - In a web browser it can be used to separate threads to handle user interactions, load website resources, and perform background operations since this allows users to continue interacting with the browser while pages and resources are loading.
   - While downloading files, a download manager can use multiple threads to download different files or separate portions of a file simultaneously. This can reduce waiting time and improve download efficiency.
   - In systems, a server can use multiple threads to handle requests from different clients since multiple users can submit things and perform tasks without requiring the server to handle every request one at a time.

3. **Differentiate between parallelism and concurrency.**
   Concurrency is where threads are worked over overlapping time. Basically, the CPU switches threads constantly where it looks like it is doing multiple work at once when in reality, it is constantly switching between multiple threads.Concurrency can be done using single core CPUs.

and is much more responsive and efficient as a result. Parallelism is where threads are actually worked by different cores at the same time. This allows it for faster speed and throughput, although it requires multiple cores or processing units to properly work. It also slows down with dependencies

Part B: Create C++ Code and show a solution that satisfies the given requirements below.



```
ILO_B.cpp > main()
#include <iostream>
#include <vector>
#include <thread>

int total = 0;
void add(int value) {
    total += value;
}

int main() {
    std::thread T1(add, 5);
    std::thread T2(add, 10);
    std::thread T3(add, 20);
    std::cout << "Initial value: " << total << std::endl;

    T1.join();
    std::cout << "After T1.join(): " << total << std::endl;
    T2.join();
    std::cout << "After T2.join(): " << total << std::endl;
    T3.join();
    std::cout << "After T3.join(): " << total << std::endl;

    return 0;
}
```

```
Problems  Output  Terminal  Ports
PS C:\Users\TIPQC\Documents\SANTOS\HOA13p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA13p1\" ; if ($?) { g++ ILO_B.cpp -o ILO_B } ; if ($?) { .\ILO_B }
Initial value: 0
After T1.join(): 35
After T2.join(): 35
After T3.join(): 35
```

- **Analysis:**
  By adding in a thread and joining all of it together, it adds up every value added in the separate threads as they are joined together which results in every thread to hold the same values.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Fabonan
Code:
Output: **Analysis:**

Santos

Code:

```cpp
#include <iostream>
#include <time.h>
#include <thread>
#include <vector>

int main(){
    // Generate random values (Assume that 100 is n for space complexity)
    int a[100];
    srand(time(0));
    for(int i = 0; i < 100; i++){
        a[i] = rand();
    }

    // Multithreading setup
    const int numThreads = 4;
    bool foundDuplicate[4] = {false, false, false, false};
    std::vector<std::thread> threads;

    // Time complexity here:
    for(int t = 0; t < numThreads; t++){
        threads.push_back(std::thread([&, t](){
            int start = t * 100 / numThreads;
            int end = (t + 1) * 100 / numThreads;

            for(int i = start; i < end; i++){
                for(int j = i + 1; j < 100; j++){
                    if(a[i] == a[j]){
                        foundDuplicate[t] = true;
                        return;
                    }
                }
            }
        }));
    }

    // Wait for all threads to finish
    for(int i = 0; i < numThreads; i++){
        threads[i].join();
    }

    // Check results
    for(int i = 0; i < numThreads; i++){
        if(foundDuplicate[i]){
            std::cout << "Returned False \n";
            return false;
        }
    }

    std::cout << "Returned True \n";
    return true;
}
```

Output:

```
Returned True
PS C:\Users\TIPQC\Documents\SANTOS\HOA13p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA13p1\" ; if ($?) { g++ ILO_C.cpp -o ILO_C } ; if ($?) { .\ILO_C }
Returned True
PS C:\Users\TIPQC\Documents\SANTOS\HOA13p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA13p1\" ; if ($?) { g++ ILO_C.cpp -o ILO_C } ; if ($?) { .\ILO_C }
Returned True
PS C:\Users\TIPQC\Documents\SANTOS\HOA13p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA13p1\" ; if ($?) { g++ ILO_C.cpp -o ILO_C } ; if ($?) { .\ILO_C }
Returned False
PS C:\Users\TIPQC\Documents\SANTOS\HOA13p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA13p1\" ; if ($?) { g++ ILO_C.cpp -o ILO_C } ; if ($?) { .\ILO_C }
Returned False
PS C:\Users\TIPQC\Documents\SANTOS\HOA13p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA13p1\" ; if ($?) { g++ ILO_C.cpp -o ILO_C } ; if ($?) { .\ILO_C }
Returned True
PS C:\Users\TIPQC\Documents\SANTOS\HOA13p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA13p1\" ; if ($?) { g++ ILO_C.cpp -o ILO_C } ; if ($?) { .\ILO_C }
Returned True
PS C:\Users\TIPQC\Documents\SANTOS\HOA13p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA13p1\" ; if ($?) { g++ ILO_C.cpp -o ILO_C } ; if ($?) { .\ILO_C }
Returned False
PS C:\Users\TIPQC\Documents\SANTOS\HOA13p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA13p1\" ; if ($?) { g++ ILO_C.cpp -o ILO_C } ; if ($?) { .\ILO_C }
Returned False
PS C:\Users\TIPQC\Documents\SANTOS\HOA13p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA13p1\" ; if ($?) { g++ ILO_C.cpp -o ILO_C } ; if ($?) { .\ILO_C }
Returned True
```

Analysis:

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion
of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.